

2026 REFERENCE EDITION · SYSTEMS ARCHITECTURE

GRAPH ENGINEERING

The Definitive Field Guide to Knowledge Graphs & Agent
Task Graphs

Prompt engineers steered words. Loop engineers steered iterations. **Graph engineers steer system topology.**

A complete, production-grade guide unifying what AI agents *remember* (the 9-stage knowledge graph pipeline) with how AI agents *work* (task graph DAGs, the diamond pattern, verifier separation, and the DeepMind × MIT stop rule).

SOURCE CURRICULUM

Southeast University (Prof.
Peng Wang)

TASK GRAPH RESEARCH

Google DeepMind × MIT
CSAIL

DISCIPLINE DOMAIN

Autonomous Agents &
GraphRAG

Table of Contents

1. Executive Foreword: The 2026 Shift to System Topology

- From Prompt Engineering to Graph Engineering
- The Two Foundational Halves: Memory vs. Execution

2. Foundations & The 9-Stage Knowledge Graph Pipeline

- Stage 1: Scope & Competency Questions
- Stage 2: Knowledge Representation Choice
- Stage 3: Ontology Modeling & Schema Design
- Stage 4: Entity Extraction (NER Ladder)
- Stage 5: Relation Extraction with Strict Validation
- Stage 6: Event Extraction & Causal Event-Logic Graphs
- Stage 7: Quality Gate & $\geq 90\%$ Precision Barrier
- Stage 8: Knowledge Fusion (Blocking, Neighborhood Matching, Merge Policy)
- Stage 9: Serving to LLMs (GraphRAG, Path Serialization, Memory Loops)

3. Task Graphs: The Execution Half of Graph Engineering

- Jobs as DAG Nodes and Eliminating Fake Edges
- The Diamond Pattern & Independent Verifier Separation
- The Stop Rule: Google DeepMind $\times$ MIT 180-Config Scaling Law
- The Human Gate & Production Guardrails

4. Complete Translated Curriculum: Southeast University Knowledge Graph Course

- Lectures 1 through 9 In-Depth Analysis
- Bridge from Classical AI/NLP to 2026 LLM Agent Workflows

5. The 9 Production Workflows & Implementation Blueprints

- Interactive Tools (/kg-tutor, /kg-scope, /kg-schema, /kg-extract, /kg-relations, /kg-events, /kg-fuse, /kg-eval, /kg-rag)

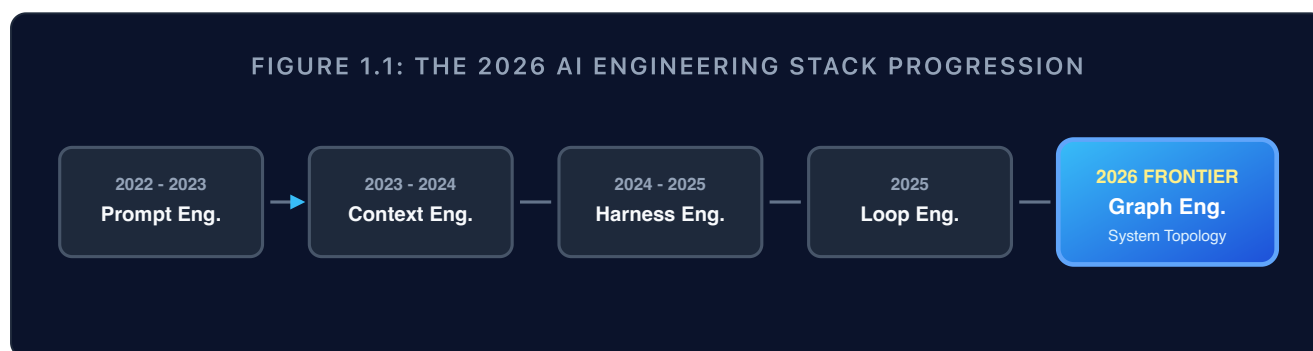
6. Agent Skill Specification (graph-engineering.skill)

- Autonomous Skill Specification & Operational Checklist

1. Executive Foreword: The 2026 Shift to Topology

Over the last four years, AI engineering progressed through distinct abstraction layers. In 2022 and 2023, developers focused on **Prompt Engineering**: tuning tokens, framing system personas, and optimizing zero-shot or chain-of-thought phrases. By 2024, the frontier shifted to **Context Engineering**: building vector databases, chunking strategies, and retrieval-augmented generation (RAG) pipelines. In 2025, the industry recognized that single completions were insufficient, introducing **Harness Engineering** (tool sandboxes, agent environments) and **Loop Engineering** (autonomous REPL loops, test-driven iteration, Ralph loops).

In 2026, the ceiling of uncoordinated loops has been reached. A model looping aimlessly degrades rapidly. Scaling autonomous agents now demands **Graph Engineering**: the discipline of designing the topological structures through which AI agents remember and work.



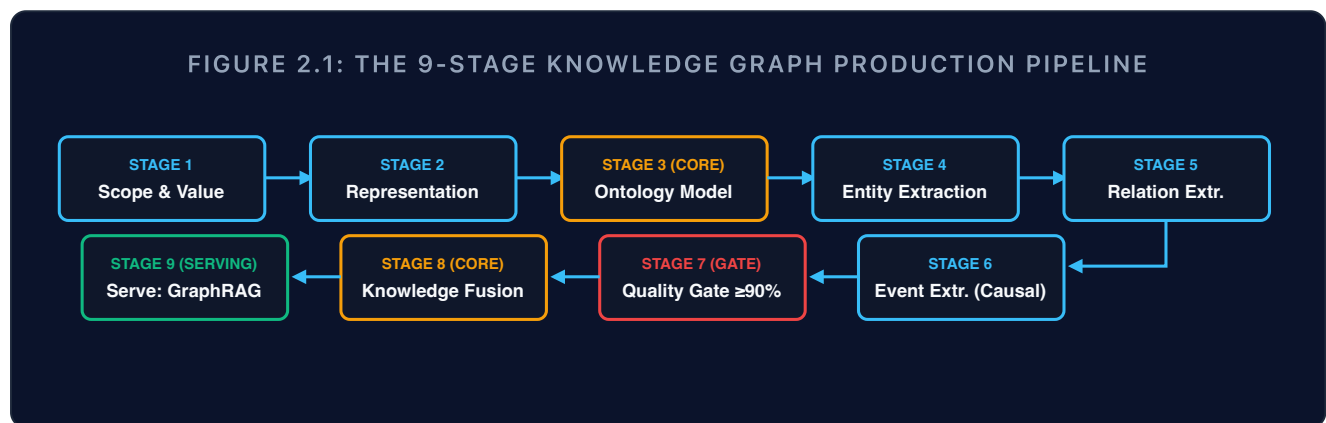
The Core Mental Model: Prompt engineers steered the model's words. Loop engineers steered its iterations. **Graph engineers steer its topology.** A knowledge graph is a *product with an explicit schema*, not an unorganized pile of triples.

Graph Engineering divides cleanly into two complementary halves:

- 1. Knowledge Graphs (What agents remember):** Nodes represent entities, events, and ground facts; edges represent typed relationships annotated with time and verifiable provenance. It follows a rigorous 9-stage engineering pipeline: Scope → Representation → Ontology → Entities → Relations → Events → Quality Gate → Fusion → Serving to LLMs.
- 2. Task Graphs (How agents work):** Nodes represent discrete jobs; edges represent strictly necessary execution data dependencies. It enforces parallel fan-out, independent verifier contexts, the Google DeepMind × MIT stop rule, and human approval gates.

2. The 9-Stage Knowledge Graph Production Pipeline

Quality in a knowledge graph is a direct function of pipeline sequencing. The most common catastrophic failure in enterprise graph projects is extracting before establishing a schema, or dumping triples into storage before entity fusion. The 9-stage pipeline guarantees structural integrity.



Stage 1: Scope & Value Test

Before writing code or instantiating a database, confirm that a graph structure mathematically beats simpler tabular or vector alternatives. A knowledge graph pays significant dividends when:

- Queries require **multi-hop traversals** (e.g., *"Find all third-party dependencies of service A maintained by engineers who contributed to incident B"*).
- Entities recur across heterogeneous documents with complex, interconnected aliases.
- The relationships themselves constitute the primary data payload (with edge weights, temporal intervals, and provenance).

Rule of Thumb: If your query workload consists of single-hop lookups or simple key-value aggregations, use PostgreSQL or ClickHouse and stop. Do not force graph complexity onto tabular data.

Stage 2: Knowledge Representation Choice

Select how facts are encoded based on query requirements and team scale:

Representation	When to Choose	Trade-offs & Overhead
Property Graph (Neo4j, Kùzu, Memgraph)	Pragmatic industry default for products, GraphRAG, and agent memory. Nodes and edges carry arbitrary key-value properties.	Lacks formal description-logic semantics. Schema consistency must be enforced at the application/pipeline layer.
RDF / OWL Triples (Jena, GraphDB, Virtuoso)	Strict standards compliance, biomedical ontologies, public interop, automated inference (subsumption, consistency checking).	Verbose; requires reification or RDF-star to attach edge metadata; steeper tooling curve.
Typed SQLite / JSON Edges (Local file / SQLite)	Small-scale (<50,000 nodes), agent-local persistent memory, zero external infrastructure dependencies.	Query expressiveness capped; multi-hop traversals degrade at high node counts.

Non-Negotiable Constraint: Decide during this stage how every fact carries **Time** (validity interval or timestamp: `valid_from`, `valid_until`) and **Provenance** (source document ID, URI, exact character span, extraction timestamp, confidence). Retrofitting provenance after fusion is impossible.

Stage 3: Ontology Modeling & Schema Design

An ontology defines the schema of the graph. Distilled from Southeast University's graduate course:

1. **Competency Questions:** Draft 10–20 concrete domain questions the graph must answer. These serve as the specification and the acceptance test suite.
2. **Core Entity Types:** Enumerate 5–15 foundational types (e.g., `Person`, `Organization`, `Service`, `Incident`). Each type requires a definition and canonical identification rule.
3. **Relation Types with Domain & Range:** Define typed edges with explicit constraints (e.g., `CONTRIBUTED_TO: Person → Service`).
4. **Entities vs. Attributes:** If a concept possesses relationships of its own, it is an Entity. If it is merely a scalar value used for filtering or display, it is an Attribute.
5. **Precise Verbs:** Use specific predicates such as `ACQUIRED`, `MAINTAINS`, `DEPENDS_ON` —never vague relations like `RELATED_TO` or `HAS_LINK`.

```
# Example Production Ontology Specification (ontology.yaml)
entities:
  Engineer:
    desc: Software engineer or contributor
    identifier: email
    attributes: [name, team, role]
  Service:
    desc: Deployable microservice or library
    identifier: service_name
    attributes: [tier, repository_url]
  Incident:
    desc: Production outage or customer-facing degradation
    identifier: incident_id
    attributes: [severity, root_cause, start_time, resolved_time]

relations:
  COMMITTED_TO:
    domain: Engineer
    range: Service
    cardinality: many-to-many
  TRIGGERED_INCIDENT:
    domain: Service
    range: Incident
    cardinality: one-to-many
  RESOLVED_BY:
    domain: Incident
    range: Engineer
    cardinality: many-to-many
```

Stage 4: Entity Extraction (NER Ladder)

Extract typed entities from raw sources following the method ladder:

1. **Rule/Dictionary Match First:** For closed vocabularies (stock tickers, country codes, internal team lists, internal service catalogs), exact string matching achieves 100% precision with zero LLM inference cost.
2. **LLM Extraction for Open Text:** Provide the ontology schema directly in the prompt. Require the model to emit the surface form, canonical form, type, exact evidence sentence, and confidence score.
3. **Provenance Logging:** Capture document URI, chunk index, character offset, and extraction model ID.

Stage 5: Relation Extraction with Strict Validation

Extract typed relationships between entities identified in Stage 4:

- **Entity Pre-condition:** Never permit relation extraction to invent entities on the fly. Relations must link entities that passed Stage 4.
- **Code-Level Domain/Range Validation:** Reject invalid edges deterministically before database insertion. For instance, if an LLM outputs `COMMITTED_T0: Service → Incident`, automatic type checking drops the edge immediately.
- **Distinguish Co-occurrence from Assertion:** Two entities appearing in the same sentence do not imply a relationship. Require verbatim assertion evidence.

Stage 6: Event Extraction & Causal Event-Logic Graphs

For dynamic domains (financial transactions, cybersecurity breaches, cloud infrastructure incidents), model events as **first-class nodes** rather than flat edges.

Concept: Causal Event-Logic Graphs: Southeast University Lecture 7 introduces graphs where nodes represent distinct events and edges represent directed causal, temporal, or conditional relationships (*Event A* → *CAUSES* → *Event B*). This enables backward root-cause analysis and forward predictive risk simulation.

Stage 7: Quality Gate (≥90% Precision Barrier)

Before running expensive fusion algorithms across the entire corpus, draw a stratified sample of 50 extracted items and evaluate:

- **Entity Precision:** Are the entities genuine and assigned to the correct ontology type?
- **Relation Precision:** Does the quoted evidence span directly support the asserted edge?

Quality Gate Threshold: If precision falls below 90%, **halt the pipeline**. Do not patch corrupted data by hand; iterate on prompt constraints and boundary rules, then re-extract. Impure extractions permanently poison downstream graph traversals.

Stage 8: Knowledge Fusion (The Make-or-Break Stage)

Heterogeneous documents create lexical variations for identical real-world entities (e.g., "AWS", "Amazon Web Services", "Amazon Cloud"). Unfused graphs fragment paths and silently break multi-hop reasoning.

1. **Blocking:** Avoid $O(n^2)$ pairwise comparisons. Group candidate entities using coarse blocking keys (acronym expansion, shared phonetic keys, high-threshold embedding similarity).

2. Layered Matching:

- *String Layer*: Normalized Levenshtein distance, Jaro-Winkler, alias dictionaries.
- *Attribute Layer*: Compatible metadata (domain names, VAT numbers, dates).
- *Structure / Neighborhood Layer*: Compare adjacent graph topology. If two "John Smith" nodes share three coauthors and a university affiliation, they represent the same individual.
- *LLM Adjudication*: Route only ambiguous borderline candidates (score 0.65–0.85) to an LLM with full neighborhood context.

3. **Reversible Merge Policy**: Retain canonical names, preserve source aliases, union non-conflicting edges, log provenance for both original entities, and maintain a rollback log.

Stage 9: Serving the Graph to LLMs (GraphRAG & Graph Memory)

Transform the structured graph into actionable intelligence for LLM agents:

- **GraphRAG Subgraph Retrieval**: Link entities mentioned in the user query to graph nodes → traverse k hops ($k \in [1, 2]$; $k > 2$ introduces noise without reranking) → serialize the extracted subgraph into clean triple lines.
- **Compact Serialization Format**: Present paths as (Entity A)–[RELATION {since: 2024}]→ (Entity B) grouped by subject. This allows LLMs to cite facts with pinpoint provenance.
- **Continuous Agent Memory Loop**: After completing a mission, an agent extracts newly discovered facts → fuses them into persistent graph storage → accesses updated topological context in subsequent sessions.

3. Task Graphs: Agent Orchestration & Execution Topology

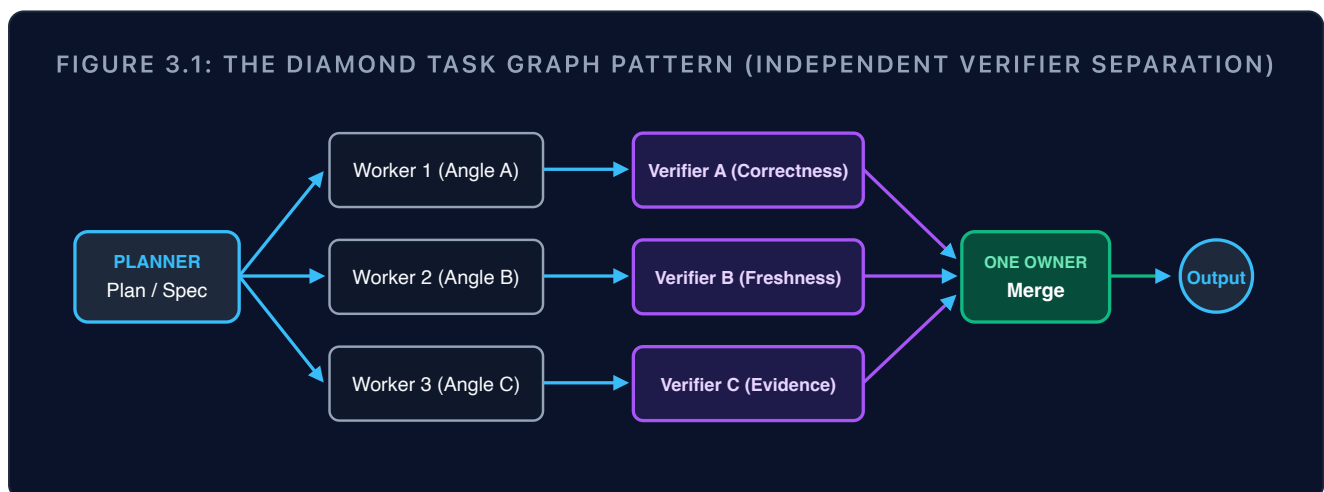
While knowledge graphs govern agent *memory*, task graphs govern agent *execution*. A task graph is a Directed Acyclic Graph (DAG) where nodes represent discrete tasks and directed edges represent dataflow prerequisites.

Eliminating Fake Edges

The most immediate performance optimization in agent pipelines is auditing every sequential step for **fake edges**. If Step B does not strictly consume the output of Step A, no dependency edge should exist. Pruning fake edges transforms sluggish sequential pipelines into parallel fan-outs, drastically reducing end-to-end latency.

The Diamond Pattern & Verifier Separation

Production multi-agent architectures converge on the **Diamond Pattern**:



Key Architecture Principle: An agent must never grade its own work inside its own context window. Autoregressive models are prone to self-justifying hallucinations. The diamond pattern executes parallel workers, passes candidate artifacts to **isolated, independent verifier contexts** with specialized rubrics (correctness, safety, freshness), and routes surviving work to a single merge owner.

The Stop Rule: Google DeepMind × MIT Empirical Scaling Law

In the landmark study *"Towards a Science of Scaling Agent Systems"* (180 controlled multi-agent configurations), researchers established decisive laws for agent coordination:

Workload Topology	Single Agent	Coordinated Multi-Agent Team	Empirical Result & Law
Splittable Workload (Independent angles, parallel research)	Baseline (1.0×)	~1.8× performance gain (+80%)	Coordinated teams excel when work can be partitioned without inter-agent communication overhead.
Sequential Workload (Step B strictly requires Step A context)	Optimal (Wins)	Degrades 39% – 70%	Every multi-agent setup loses. Context handoffs cause catastrophic context degradation.
Error Propagation (Uncoordinated vs. Coordinated Merge)	Baseline	Uncoordinated: 17.2× error explosion Single Owned Merge: 4.4×	Decentralized multi-agent consensus amplifies hallucinations. A single dedicated merge owner is mandatory.

The DeepMind × MIT Stop Rule: More agents is not a strategy. The shape of the work decides. If a problem is inherently sequential, assign it to a single competent agent. If it naturally splits, fan out to parallel workers, verify in separate contexts, and collapse back through one merge coordinator.

The Human Gate & Production Guardrails

- **Placing Human-in-the-Loop Gates:** Place approval checkpoints strictly where mistakes are expensive, catastrophic, or irreversible (e.g., executing financial transactions, modifying live database schemas, dropping resources, sending public communications). Do not place gates on non-destructive read operations.
- **Production Guardrails:**
 1. **Iteration Ceilings:** Enforce hard maximum turn caps on every loop to prevent runaway token expenditure.
 2. **Single Writer Rule:** No two concurrent worker nodes may mutate the same file or state record simultaneously.

3. **Deterministic Topology:** Workflow routing is encoded in static code/DAG definitions; LLMs populate node payloads, not the execution graph.
4. **Concurrency Limits:** Enforce strict bounds on child agent spawning.

4. Academic Grounding: Southeast University Curriculum

The knowledge-graph half of this discipline is grounded in Southeast University's graduate curriculum "*Knowledge Graphs*", delivered by Prof. Peng Wang (Southeast University School of Computer Science & Engineering, 4.4K ☆ open syllabus since 2019). The 2025/2026 edition unifies classical knowledge engineering with modern generative AI:

Lecture	Academic Curriculum & Topics	2026 AI Agent Application
Lecture 1	Theory, Technology, Practice & Cognitive Challenges: Evolution from semantic networks, expert systems, and the Semantic Web to modern Knowledge Graphs. Fundamental bottlenecks in acquisition, noise, and consistency.	Defines the role of structured knowledge in grounding stochastic LLM agents and preventing factual hallucination.
Lecture 2	Knowledge Representation: Formal systems: Frames, Conceptual Graphs, Description Logics (DL), RDF/RDFS/OWL semantic primitives, and representation learning (KG embeddings).	Guides schema selection between property graphs, typed SQLite memory, and formal OWL models.
Lecture 3	Knowledge Modeling: Ontology engineering methodologies, competency question scoping, schema induction, Protégé tooling, and conceptual taxonomy design.	Provides the foundation for prompt-driven ontology design (<code>ontology.yaml</code>) and domain constraint enforcement.
Lecture 4	Extraction Methodologies & Source Routing: D2R mapping for structured tables, wrapper induction for web tables, and NLP pipelines for unstructured prose.	Establishes the source routing rule: never use expensive LLM inference on structured SQL data.
Lecture 5	Entity Recognition Evolution: Transition from rule dictionaries → HMM/CRF → BiLSTM-CRF → Pretrained BERT → Few-shot In-Context Extraction.	Synthesizes the NER ladder: dictionaries for closed sets, LLMs with evidence spans for open domains.
Lecture 6	Relation Extraction: Pattern-based extraction, distant supervision noise handling, open information extraction (Open IE), and neural relation classification.	Enforces domain/range validation to eliminate hallucinated edges between incompatible entity types.
Lecture 7	Event Extraction & Event-Logic Graphs: Triggers, argument roles, Huawei telecommunications/finance case studies, and causal event-logic graphs.	Enables incident analysis, automated post-mortems, and temporal event reasoning in autonomous agents.
Lecture 8	Large-Scale Knowledge Fusion: Schema alignment, blocking strategies, multi-attribute similarity metrics, graph neighborhood matching, and deduplication.	Prevents graph fragmentation by reconciling entity mentions across disparate data silos.
Lecture 9	KG × LLM Integration (Bidirectional): <i>KG for LLM</i> (GraphRAG, grounding, hallucination reduction) & <i>LLM for KG</i> (schema induction, extraction, adjudication).	Underpins GraphRAG architectures and continuous graph-as-memory cycles for long-running agents.

5. The 9 Production Workflows & Implementation Blueprints

The following nine production workflows are modular prompt blueprints designed for engineering teams and autonomous agents. Each workflow operates as an independent stage or chains sequentially into an end-to-end graph deployment.

Tool 1: `/kg-tutor` — The Interactive Academic Tutor

Purpose: Walks engineers through the complete Southeast University curriculum interactively, using their specific project domain as the running example.

```
You are teaching me Southeast University's graduate Knowledge Graph course.
I want to finish able to build one, not merely describe one.
```

HOW YOU RUN THIS

```
Ask me three things, then wait for my answers:
```

1. What I'm building, or want to build
2. My level: never touched one / used a graph database / read the papers
3. Hours per week I actually have

```
Then propose a route through the modules and let me approve it. Never teach two modules :
Per module: explain the idea in plain terms using MY domain as the running example—never
movies-and-actors. Name the one mistake beginners make here. Then give me a single build
Do not continue until I show output. When I do, critique it: tell me what breaks at 100x
```

THE MODULES

- 01 concepts – what a KG is, and when it is the wrong tool
- 02 representation – semantic networks, frames, description logic, embeddings
- 03 ontology – schema design, domains and ranges (hardest, most durable)
- 04 extraction – routing sources by type
- 05 entities · 06 relations · 07 events
- 08 fusion – deduplication, alignment, blocking
- 09 embeddings – TransE family, link prediction evaluation
- 10 KG x LLM – GraphRAG, grounding, models building graphs

WHAT YOU MUST NOT DO

```
Do not teach 2016 methods as current practice. Feature-engineered NER and translation emb
literacy, not tooling. Do not let me skip 03 or 08; that is where real projects die. If r
not actually need a graph, tell me in module 01 and stop the course.
```

Tool 2: `/kg-scope` — Domain Boundary & Competency Specification

Act as a knowledge graph architect. I want to model a domain before writing any code.

Domain: [DESCRIBE IN 2 SENTENCES]

What I want to answer with it: [3 REAL QUESTIONS]

Return:

1. 8–12 entity types, each with 3–5 key attributes and what uniquely identifies an instance
2. 5–8 relation types as (subject type, predicate, object type) with cardinality
3. My 3 questions rewritten as traversals over those types
4. Anything my questions need that the schema cannot answer, and what is missing

Do not write code. If a question needs aggregation rather than traversal, say so—that is not a graph.

Tool 3: `/kg-schema` — Protégé-Ready Ontology Modeling

Act as an ontology engineer. Turn this draft schema into a real ontology.

Draft: [PASTE YOUR /kg-scope OUTPUT]

Return:

1. A class hierarchy with explicit subclass relations, no more than 3 levels deep
2. Every property with domain, range, and whether it is functional or inverse-functional
3. Turtle (.ttl) serialization ready to load into Protégé or GraphDB
4. Every modeling decision where you chose between two defensible options, and why

Reuse schema.org or existing standard vocabularies for generic concepts—only mint new IRIs for what is specific to my domain. Flag anything modeled as a class that should have been

Tool 4: `/kg-extract` — Source Routing & Extraction Architecture

Act as an extraction engineer. Design the pipeline before I build it.

Sources: [LIST THEM – e.g. 400 PDFs, a Postgres table, scraped HTML]

Target schema: [PASTE /kg-schema OUTPUT]

Return:

1. Split sources into structured / semi-structured / unstructured, and the extraction method (the first two must not use an LLM)
2. For the unstructured set: the prompt, the output JSON schema, and chunking strategy
3. The 5 failure modes most likely for this specific data, with detection checks for each
4. A 50-document hand-check protocol: sampling method, recording schema, and stopping the

Do not propose fine-tuning until the prompted baseline has a measured error rate.

Tool 5: `/kg-relations` — Provenance-Constrained Relation Extraction

Act as a relation extraction engineer.

Schema relations: [PASTE THEM]

Corpus: [DESCRIBE IT]

Return:

1. A prompt that emits only typed triples valid against my schema, each with a confidence score and verbatim evidence span
2. A distant-supervision baseline: which existing table or list I can align to text to get training pairs for free, and the noise that introduces
3. Rejection rules—triples to drop before they ever reach the graph
4. How to benchmark the two approaches against each other on 100 sample sentences

Every triple carries provenance. A triple with no evidence span is a hallucination with confidence 0.

Tool 6: `/kg-events` — Causal Event-Logic Schema Extraction

```
Act as an event extraction engineer. I want a graph of things that happened, not things that are.

Domain and corpus: [DESCRIBE]

Return:
1. An event type schema: trigger, arguments and their roles, temporal anchor
2. The extraction prompt, one record per event, with argument spans
3. The edges between events—causal, temporal, conditional—and how to distinguish "reported" from "merely co-occurred"
4. Storage design so a query can walk a chain backwards from an outcome

Keep event nodes separate from entity nodes. Never collapse a cause into an attribute.
```

Tool 7: `/kg-fuse` — Entity Resolution & Reversible Merge Policy

```
Act as an entity resolution engineer. My graph has duplicates.

Entity type and volume: [e.g. 40,000 company records]
Available fields: [LIST THEM]

Return:
1. A blocking strategy to avoid n-squared comparisons, with expected candidate reduction
2. Match function: fields, similarity measures, weights, and decision threshold
3. Review band—the score range where a human decides instead of the machine
4. Merge policy: on conflict, which source wins, and what survives as an alias rather than a new node
5. 10 hard edge cases from my field list where naive matching fails

Merges must be reversible. Specify exactly what audit log is required to undo any merge.
```

Tool 8: `/kg-eval` — Skeptical Knowledge Graph Reviewer

Act as a skeptical reviewer of my knowledge graph.

What I built: [DESCRIBE]

Numbers I'm about to claim: [PASTE THEM]

Return:

1. Precision and recall at the triple level—how to sample and estimate them with a stated interval, not a vibe
2. Where my test set leaks into my training or prompt-development set
3. If reporting link prediction: whether the filtered setting was used, and what a trivial baseline is
4. The three claims a reviewer attacks first, and the empirical experiment that defends them

Assume my numbers are inflated until the sampling method proves otherwise.

Tool 9: `/kg-rag` — GraphRAG vs. Vector Retrieval Engine

Act as a retrieval engineer. Wire my graph into an agent and prove it beats vector search.

Graph: [DESCRIBE]

Question types: [3 EXAMPLES]

Return:

1. Retrieval strategy per question type—entity lookup, k-hop traversal, subgraph extraction, or plain vector. State which questions do not need the graph at all
2. Subgraph serialization strategy to fit LLM context without blowing window limits
3. Vector-only baseline over the same source text
4. An evaluation set of 30 questions written before either system runs, with an answer key and the metric that separates them

If the graph does not win on multi-hop questions, it does not earn its maintenance cost.

6. Agent Skill Specification: graph-engineering.skill

The complete graph engineering skill integrates directly into autonomous agent frameworks (Claude Code, Cursor, Codex, OpenClaw). It provides teaching protocols and automated execution machinery.

Skill Metadata & Interface

```
---
name: graph-engineering
description: Teaches an agent graph engineering—both halves. Knowledge graphs (ontology &
  entity/relation/event extraction, fusion, GraphRAG/memory serving; distilled from South
  University's graduate Knowledge Graph course, npubird/KnowledgeGraphCourse, 4.4K stars)
  graphs (agent orchestration—parallel fan-out, verifier separation, the stop rule, human
  Use when asked to build a knowledge graph, extract entities/relations from text, design
  ontology, dedupe/merge entities, add graph memory or GraphRAG to an agent, orchestrate
  multi-agent workflows as a graph, or LEARN graph engineering.
---
```

Teaching Mode Protocol

When invoked in educational mode, the agent enforces four strict pedagogical constraints:

1. **Domain Anchoring:** Request a single real project or dataset from the user at the start. Use that project as the consistent running example throughout every stage.
2. **Mandatory Visual Artifacts:** Produce visual diagrams at every stage (Mermaid flowcharts, entity-relationship diagrams, or self-contained interactive HTML pages).
3. **Stage-Gated Progression:** Never teach multiple pipeline stages in a single message. Conclude each stage with a practical assignment (e.g., *"Write 3 competency questions for your project"*) and await completion before progressing.
4. **Final Synthesis:** Assemble the accumulated artifacts into a production-ready `ontology.yaml` and task graph upon completion.

Enterprise Deployment Checklist

Phase	Verification Milestone	Status
Scoping	10–20 Competency questions documented and proven multi-hop	[✓] MANDATORY
Ontology	<code>ontology.yaml</code> validated in Protégé; precise verbs assigned	[✓] MANDATORY
Extraction	NER & RE prompts include verbatim evidence spans and type constraints	[✓] MANDATORY
Quality Gate	50-doc hand check achieves $\geq 90\%$ precision on entities and relations	[✓] GATEWAY
Fusion	Blocking keys configured; neighborhood matching active; rollback log enabled	[✓] MANDATORY
Serving	GraphRAG multi-hop retrieval benchmarked against vector-only baseline	[✓] GATEWAY
Task Graphs	Fake edges eliminated; diamond verifiers isolated; DeepMind stop rule applied	[✓] MANDATORY

Final Takeaway: High-reliability AI agent systems are fundamentally systems of structured relationships. By uniting the 9-stage knowledge graph pipeline for agent memory with diamond task graphs for execution, engineering teams transition from probabilistic prompting to deterministic, scalable cognitive architectures.